
**Reinforcement Learning for UAV Path Optimization in Multi-Layer NTN with
ICAS Constraints using DQNN**

Project Report Submission

Name: BHAVISYA JAIN & SUJAL JAIN

Roll No. 23UCS555 & 23UCS719

PHASE 1 : Understanding the Problem and Objectives

Objectives:

- Understand the architecture of NTN involving UAVs, HAPs, and LEO satellites.
- Explore the ICAS paradigm: how UAVs perform both sensing and communication.
- Learn reinforcement learning principles, with a focus on DQNN and its variants.

Deliverables:

- A written conceptual framework outlining the environment, constraints, and goals.
- Familiarity with relevant IEEE papers and benchmarks.

1. Project Overview

This project, “**Reinforcement Learning for UAV Path Optimization in Multi-Layer NTN with ICAS Constraints using DQNN**,” aims to develop an intelligent path-planning framework for **Unmanned Aerial Vehicles (UAVs)** operating within **Non-Terrestrial Networks (NTNs)** during emergency or **natural-disaster** situations.

When terrestrial communication systems are disrupted, UAVs equipped with **Integrated Communication and Sensing (ICAS)** can rapidly re-establish links, scan affected zones, and relay critical information.

The goal is to train these UAVs through **Deep Reinforcement Learning (DRL)** so they can autonomously navigate dynamic 3-D environments while maintaining strong connectivity, efficient sensing, and low energy use.

2. Scenario and Problem Being Solved

In the aftermath of a **natural disaster** such as a flood, cyclone, or earthquake:

1. **Infrastructure Failure:** Ground communication networks collapse, with towers, fiber links, and power lines damaged.
2. **Limited Accessibility:** Roads become blocked by debris or water, restricting rescue vehicle movement.
3. **Loss of Connectivity:** Emergency teams struggle to coordinate without reliable signals or live situational data.

To address this, a **UAV equipped with ICAS (Integrated Communication and Sensing)** is deployed to:

- **Restore temporary network coverage** between ground users, HAPs, and satellites.
- **Capture situational data** (maps, thermal imagery, and hotspots) for first responders.
- **Operate safely** around obstacles and no-fly zones while conserving limited battery power.

The UAV is trained in a **simulated 3-D environment** using **DQNN-based Reinforcement Learning**, where it learns to:

- Stay within **high-SNR coverage zones**,
- **Visit sensing points** efficiently,
- **Avoid obstacles and unsafe areas**, and
- **Optimize energy usage** for extended flight time.

This approach enables **faster network restoration**, **quicker area assessment**, and **better coordination** among rescue teams in disaster-affected zones.

3. Objectives

- Understand the **NTN architecture** (UAV + HAP + LEO satellite layers).
 - Study the **ICAS** concept of joint sensing + communication.
 - Learn **RL fundamentals** and apply **Dueling Double DQN (DQNN)**.
 - Formulate the **state, action, and reward** design for UAV path learning.
 - Prepare the **conceptual framework** for later simulation and training.
-

4. Architecture of Non-Terrestrial Networks (NTNs)

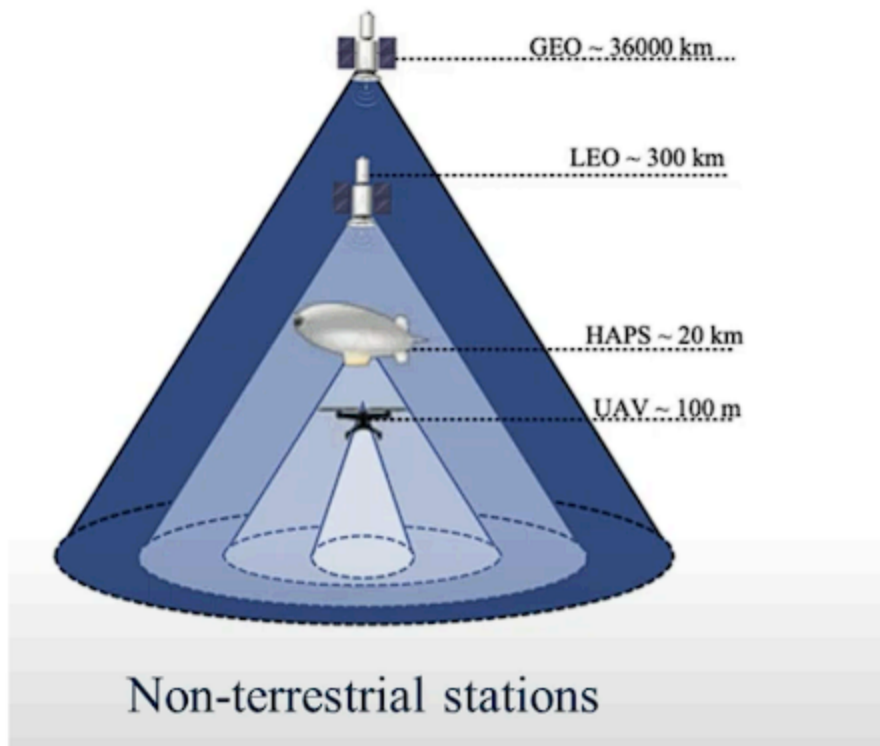
Non-Terrestrial Networks (NTNs) extend terrestrial communication by incorporating aerial and space layers that ensure wide coverage when ground systems fail.

Layer	Altitude / Type	Key Role
LEO Satellites	200 – 2 000 km	Provide high-bandwidth, low-latency backhaul links.
HAPs (High-Altitude Platforms)	~20 km	Act as mid-layer relays; rapid deployment over affected regions.
UAV Layer	Below 5 km	Handles local coverage and sensing operations.

3GPP NTN Models:

- *Transparent RAN* – satellites act as signal relays.
- *Regenerative RAN* – onboard processing and forwarding.
- *Multi-Connectivity* – integration of LEO, HAP, and terrestrial nodes for resilience.

This layered design allows the UAV to dynamically select backhaul links and maintain connectivity even when ground infrastructure is compromised.



5. Understanding ICAS – Integrated Communication and Sensing

Integrated Communication and Sensing (ICAS) equips UAVs to perform two functions simultaneously:

1. **Communication Relay:** Maintain link quality with users, HAPs, or LEO satellites.
2. **Environmental Sensing:** Gather visual, thermal, or RF data for situation assessment.

Key Challenges and Trade-offs:

- **SNR vs Altitude:** Higher altitude improves link stability but reduces sensing resolution; lower altitude does the opposite.
- **Energy Management:** Limited battery must support flight, sensing, and transmission.
- **Obstacle Avoidance:** Buildings and terrain require real-time path correction.
- **Beam Alignment:** Directional antennas demand attitude stability.
- **Dynamic Adaptation:** Changing weather and network conditions require online decision updates.

ICAS thus turns the UAV into a **self-optimizing node** that balances sensing, communication, and energy through learned policies.

6. Reinforcement Learning Algorithms for UAV Path Optimization

This section explains how reinforcement learning enables the UAV to learn autonomous flight behavior for coverage restoration and sensing after a natural disaster. The algorithms progress from basic Q-Learning to Deep Q-Network (DQN), Double DQN, and Dueling DQN, which together form the DQNN agent used in this project.

6.1 Deep Reinforcement Learning Basics

In reinforcement learning (RL), an **agent** (here, the UAV) interacts with an **environment** (the 3-D NTN space) to learn a control policy.

At every time step, the UAV observes a **state (s)**, takes an **action (a)**, and receives a **reward (r)** from the environment.

This creates a sequence:

$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots$

The goal is to maximize the total **cumulative reward** over time.

Future rewards are discounted by a factor **gamma** ($0 \leq \text{gamma} < 1$), so the total return is

$G = r_1 + \text{gamma} * r_2 + \text{gamma}^2 * r_3 + \dots$

The process is modeled as a **Markov Decision Process (MDP)**, where the next state and reward depend only on the current state and action.

In this project, the UAV's **state** may include its position (x,y,z), battery level, link quality (SNR), and area coverage achieved so far.

The **actions** correspond to UAV movements such as north, south, up, down, or hover.

The **reward function** reflects project goals: maintaining good signal strength, expanding sensed coverage, avoiding collisions, and conserving energy.

An example reward equation is:

$R = \alpha * (\text{SNR} > \text{threshold}) + \beta * (\text{new_area_covered}) - \text{gamma} * (\text{collision_penalty}) - \delta * (\text{energy_used})$

where alpha, beta, gamma, and delta are tunable weight parameters.

6.2 Q-Learning

Q-Learning is a value-based RL algorithm that learns the expected total reward for taking an action from a given state.

The update rule is:

$$Q(s,a) = Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

The **epsilon-greedy policy** controls exploration:

- With probability epsilon, a random action is selected (exploration).
- With probability (1 - epsilon), the best known action is chosen (exploitation).

While effective in small discrete grids, Q-Learning becomes infeasible for high-dimensional 3-D environments like ours.

Pseudocode (Q-Learning)

Algorithm 1: Pseudocode: Tabular Q Learning

Input: Initial ϵ , Final ϵ , ϵ -decay rate, learning_rate α , Reward Discounting Factor γ , Number of Steps n_steps ,

Environment

$Q_{mat} = \{\}$, $k = 0$

Reset Environment to get initial observation x_k ;

while $k < n_steps$ **do**

$u_k \leftarrow \begin{cases} \arg \max_u Q(x_k, u) & \text{probability } 1 - \epsilon \\ \text{Random action} & \text{probability } \epsilon \end{cases}$

 Step environment with action u_k and get x_{k+1} and reward r_k ;

if x_{k+1} *is terminal state* **then**

$Q_{mat}(x_k, u_k) \leftarrow Q_{mat}(x_k, u_k) + \alpha(r_k - Q_{mat}(x_k, u_k))$

$x_k \leftarrow$ Reset Environment

end

else

$Q_{mat}(x_k, u_k) \leftarrow Q_{mat}(x_k, u_k) + \alpha(r_k + \gamma \max_u Q_{mat}(x_k, u) - Q_{mat}(x_k, u_k))$;

end

$\epsilon \leftarrow \epsilon \times (\text{decay rate})$;

$k \leftarrow k + 1$;

end

6.3 Deep Q-Network (DQN)

To handle large or continuous state spaces, Q-Learning is extended with a neural network that approximates $Q(s,a)$.

This forms a **Deep Q-Network (DQN)**.

Training steps:

1. Compute target value
 $\text{target} = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$
2. Compute loss
 $\text{loss} = (Q_{\text{main}}(s,a) - \text{target})^2$
3. Update the network by minimizing this loss using sampled experiences.

Key mechanisms for stability:

- **Replay Buffer:** stores past transitions (s,a,r,s') and samples them randomly.
- **Target Network:** a slowly updated copy of the main network to stabilize training.
- **Epsilon Decay:** gradually lowers exploration as learning progresses.

Pseudocode (DQN)

Algorithm 1 Deep Q-learning with Experience Replay

```
Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights
for episode = 1,  $M$  do
  Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequenced  $\phi_1 = \phi(s_1)$ 
  for  $t = 1, T$  do
    With probability  $\epsilon$  select a random action  $a_t$ 
    otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
    Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$ 
  end for
end for
```

6.4 Double DQN (DDQN)

DQN can overestimate Q-values because it uses the same network for action selection and evaluation.

Double DQN (DDQN) fixes this using two networks: one to choose the action and another to evaluate it.

$a_star = \operatorname{argmax}_{a'} Q_main(s', a')$

$target = r + \gamma * Q_target(s', a_star)$

$loss = (Q_main(s, a) - target)^2$

This separation reduces bias and improves stability, especially in noisy post-disaster conditions.

Pseudocode (DDQN)

Algorithm 1. High level overview of Double Deep Q Network model

```
1: Initialize: primary network  $\theta$ 
2: Initialize: target network  $\theta^-$ 
3: Initialize: reply buffer  $\mathcal{D}$  with size  $\mathcal{N}$ 
4: while Training episodes not complete do
5:   Reset
6:   while not in terminal state do
7:     Observe state  $s_t$ 
8:     With probability  $\varepsilon$  select a random action  $a_t$ 
9:     Otherwise select  $a_t = \operatorname{argmax}_a Q(s_t, a; \theta)$ 
10:    Execute action  $a_t$  and observe next state  $s_{t+1}$ , receive reward  $r_t = R(s_t, a_t)$ 
    and termination info
11:    Add (state  $s_t$ , action  $a_t$ , reward  $r_t$ , next state  $s_{t+1}$ ) to reply buffer  $\mathcal{D}$ 
12:    Sample random minibatch  $(s_j, a_j, r_j, s_{j+1})$  from  $\mathcal{D}$ 
13:    If episode terminates set  $Y_j = r_j$  otherwise
14:     $Y_j = r_j + \gamma Q(s_{j+1}, \operatorname{argmax}_a Q(s_{j+1}, a; \theta_j); \theta_j^-)$ 
15:    Call DPSGD
16:    Perform a gradient descent step
17:    Every  $\mathcal{C}$  step update target network parameters
18:  end while
19: end while
20: Output : trained DDQN
```

6.5 Dueling DQN

Standard DQN learns a single Q-value per action, but does not clearly separate how good the **state** itself is.

Dueling DQN splits the network into two branches:

- **V(s)**: the value of the current state.
- **A(s,a)**: the advantage of each possible action.

They are combined as:

$$Q(s, a) = V(s) + (A(s, a) - \text{average}_a A(s, a))$$

This helps the UAV quickly recognize valuable regions (for example, strong signal zones or high-coverage altitudes) even before finding the best move.

6.6 Combined DQNN Approach Used

Our UAV agent applies a **Dueling Double DQN (DQNN)** framework enhanced with:

- **Prioritized Replay:** gives higher sampling weight to rare or critical experiences such as near collisions or link recovery.
- **Action Masking:** blocks invalid actions that lead into obstacles, no-fly areas, or out-of-bounds cells.
- **Multi-Step Returns:** uses the sum of rewards over multiple steps for faster convergence.
- **Curriculum Training:** starts with simple open maps and gradually introduces complex conditions such as dynamic obstacles, reduced energy, and intermittent satellite coverage.

This design enables the UAV to learn a robust navigation policy that balances communication quality, sensing coverage, and power efficiency — crucial for safe and effective operations in post-disaster NTN environments.

PHASE 2 : Simulation Environment Design

Objectives:

- Design a 3D simulation environment with multiple layers representing ground, UAV, HAP, and LEO nodes.
- Create synthetic datasets of ground users, obstacles, and communication channels.

Deliverables:

- A modular environment (UAVEnv) with reset() , step() , and render() functions.
- Simulation scripts to generate user locations, obstacles, and channel conditions.

Environment Components:

- State Space (s)
- UAV position (x, y, z)
- Battery level
- Link SNR (signal-to-noise ratio)
- Sensed area coverage ratio
- Distance to next waypoint/goal
- Proximity to obstacles
- Action Space (a)
- Move directions: { $\uparrow$, $\downarrow$, $\leftarrow$, $\rightarrow$, $\nearrow$, $\searrow$, hover}
- Reward Function (R): $R = +\alpha * (\text{SNR} > \text{threshold}) + \beta * (\text{new area covered}) - \gamma * (\text{collision risk}) - \delta * (\text{energy consumption})$ Where α , β , γ , δ are tunable weights.

Tools and Frameworks: Python OpenAI Gym (for environment structure) ; Matplotlib / PyBullet / AirSim (for 3D visualization)

2.1 Overview

This phase develops a **simulation testbed** for training and evaluating the UAV in **Non-Terrestrial Network (NTN)** and **Integrated Communication and Sensing (ICAS)** conditions.

The environment models a **disaster-affected region** where the UAV must maintain network connectivity, avoid obstacles, and provide user coverage.

A **2-D heatmap view** of the grid shows signal strength (SNR), obstacles, users, and the UAV's flight path.

2.2 Coordinate Space

- The environment is modeled as a **3-D grid of size $(20 \times 20 \times 5)$** .
 - The UAV operates primarily at altitude **$z = 1$** .
 - Each grid cell corresponds to a valid flight location.
 - **Obstacles** are randomly placed cells that restrict movement.
 - **Users** are distributed over the grid to represent communication demand.
 - The **start** and **goal** positions are selected from free cells before each episode.
-

2.3 Multi-Layer Network and SNR Map

- A **synthetic SNR map** defines link quality across the grid, with normalized values between 0 and 1.
- These SNR variations emulate NTN layers (ground BS, UAV, HAP, and LEO).
- The UAV references this map continuously to maintain communication quality.

2.4 UAV State Space

At every step, the UAV observes the following normalized state features:

Feature	Description
(x, y, z)	Current position coordinates
Battery	Remaining energy (0–1)
SNR	Signal strength at current cell
Coverage Ratio	Fraction of area visited
Distance to Goal	Normalized Euclidean distance
Obstacle Proximity	Relative distance to nearest obstacle

This state vector provides both environmental and mission-specific context for decision-making.

2.5 Action Space

A **7-action** discrete space governs UAV movement:

ID	Action	Description
0	Up	Move north (+Y)
1	Down	Move south (-Y)
2	Left	Move west (-X)
3	Right	Move east (+X)
4	Forward-Diagonal (↗)	Move north-east (+X,+Y)
5	Backward-Diagonal (↘)	Move south-east (+X,-Y)
6	Hover	Stay in place

Invalid moves beyond grid limits or into obstacles are masked automatically.

2.6 Reward Function

The UAV learns from a **multi-objective reward** combining communication, sensing, safety, and energy criteria:

$$\text{Reward} = \alpha * \text{SNR_condition} + \beta * \text{Area_coverage_gain} - \gamma * \text{Collision_penalty} - \delta * \text{Energy_used}$$

Where

- α – reward for maintaining $\text{SNR} > 0.6$
- β – reward for covering new cells (sensing progress)
- γ – penalty for proximity to obstacles or collisions
- δ – penalty for energy consumption

Typical weights: $\alpha = 1.0$, $\beta = 2.0$, $\gamma = 4.0$, $\delta = 1.0$.

2.7 Code Structure

`data_generator.py`

- Creates `users.npy`, `obstacles.npy`, and `channel_map.npy` datasets.
- Stores them in the `/data/` directory for repeatable environment loading.

`uav_env.py`

- Defines `UAVEnv` class based on the OpenAI Gym template.
- Implements `reset()`, `step()`, `render()`, and `render_map()` methods.
- Handles motion updates, state formation, reward calculation, and termination.

`main.py`

- Runs sample episodes using random actions to test the environment.
- Prints step-wise position and battery logs and renders the map periodically.

requirements.txt

- Dependencies: NumPy, Gym, Matplotlib.
-

2.8 Sample Execution Log

Step 1: Position [0 0 1], Battery 0.99

Step 2: Position [0 1 1], Battery 0.98

Step 3: Position [0 2 1], Battery 0.97

...

Step 80: Position [5 11 1], Battery 0.20

The UAV begins from (0, 0, 1) and progressively navigates toward its goal, avoiding obstacles and staying within strong signal zones.

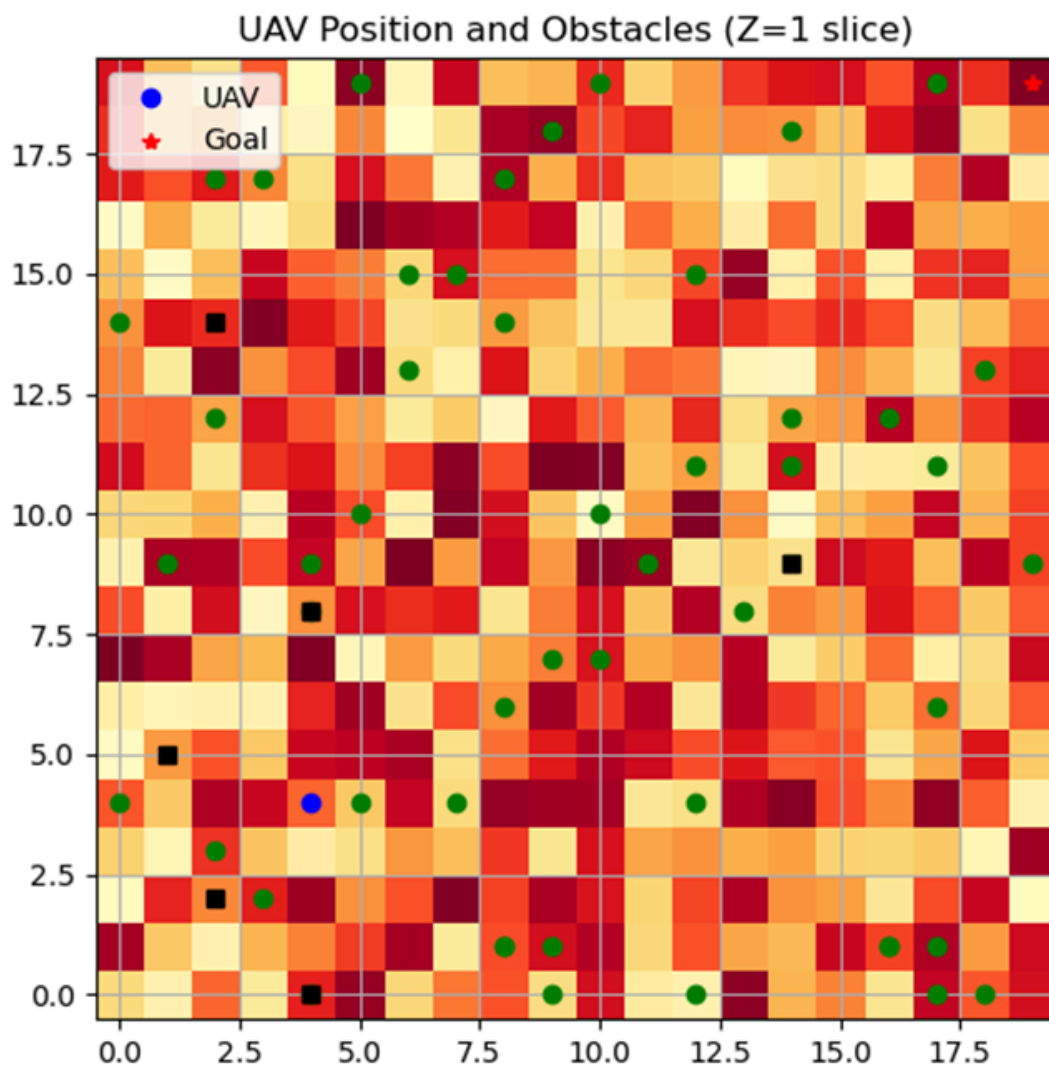
2.9 Visualization Snapshot

The environment automatically renders every 20 steps.

The map shows:

- UAV position (blue dot)
- Goal (red star)
- Obstacles (black squares)
- Users (green dots)
- SNR distribution (background heatmap)

This visualization helps verify environment logic and UAV path behavior in real time.



PHASE 3 : DQNN Agent Design and Implementation

Objectives:

- Implement the DQNN agent with support for Dueling and Double DQN architectures.

Network Architecture:

- Input layer: size = length of state vector
- Hidden layers: 2–3 Dense layers (64–128 units), ReLU activation
- Output layer: Q-values for each action
- Target Network: for stable learning

Advanced Features:

- Experience Replay Buffer
- ϵ -Greedy Exploration Strategy
- Target Network Update (e.g., every 100 steps)
- Optional: Prioritized Experience Replay

Libraries:

- PyTorch or TensorFlow
 - NumPy, Matplotlib
-

3.1 Objectives

The objective of this phase was to design and implement a **Deep Q-Network (DQN)**-based learning agent for UAV path optimization in the NTN + ICAS environment built in Phase 2.

The agent was enhanced into a **Dueling Double DQNN** structure that supports stable and efficient learning.

Specific goals:

1. Implement a DQNN agent supporting **Double** and **Dueling** architectures.
 2. Integrate key RL techniques: **Experience Replay**, **ϵ -Greedy exploration**, and **Target Network update**.
 3. Ensure compatibility with the continuous 2-D UAV environment ([uavenv.py](#)).
 4. Build a modular, reproducible training workflow using PyTorch, NumPy, and Matplotlib.
-

3.2 Network Architecture

Layer	Description	Parameters
Input Layer	Accepts UAV state vector (x, y, z_norm, battery, SNR, coverage, distance to goal, obstacle proximity).	size = length of state vector (≈ 8)
Hidden Layer 1	Fully connected, ReLU activation	128 neurons
Hidden Layer 2	Fully connected, ReLU activation	64 neurons
Dueling Heads	Two parallel streams: Value $V(s)$ and Advantage $A(s,a)$. Combined as $Q(s, a) = V(s) + (A(s, a) - \text{mean}(A(s, :)))$	—
Output Layer	Produces Q-values for each of the 7 discrete actions	7 neurons

- **Optimizer:** Adam (lr = 0.001)
 - **Loss:** Mean Squared TD-Error
 - **Discount Factor (γ):** 0.95
 - **Batch Size:** 64
 - **Target Network Update:** every 100 steps
- This architecture balances representational power with training stability.

3.3 Algorithmic Features

Double DQN Mechanism

To reduce overestimation bias:

$a^* = \operatorname{argmax}_a Q_{\text{main}}(s', a)$ # action chosen by main net

$\text{target} = r + \gamma * Q_{\text{target}}(s', a^*)$ # value estimated by target net

Dueling Network Head

Separates the **state value** $V(s)$ and the **relative advantage** $A(s,a)$:

$Q(s,a) = V(s) + (A(s,a) - \operatorname{mean}(A(s,:)))$

This helps the network distinguish whether the UAV's *state* itself is good (e.g., in a strong SNR zone) even if all actions look similar.

Experience Replay Buffer

Stores transitions (s,a,r,s',done) for randomized mini-batch training, reducing correlation between consecutive updates.

ϵ -Greedy Exploration

Balances exploration and exploitation:

if $\text{random}() < \epsilon$: take random action

else: take $\operatorname{argmax}_a Q(s,a)$

ϵ decays from 1.0 $\rightarrow$ 0.05 over training

Target Network Update

A second copy of the DQNN is synchronized with the main network every 100 steps for stable TD-targets.

3.4 Implementation Summary

dqnn_agent.py

- Implements the `DQNNAgent` class containing:
 - Dueling network (`DuelingQNet`)
 - Experience Replay Buffer

- Double DQN target computation
- Epsilon scheduling and target sync

- Core update:

$$\text{loss} = (Q_{\text{main}}(s,a) - [r + \gamma * Q_{\text{target}}(s',a^*)])^2$$

train_dqnn.py

- Initializes environment (`UAVEnv`) and agent.
- Trains for ≈ 1500 episodes $\times$ 300 steps.
- Logs episode reward, collision count, coverage ratio, success rate, and ϵ decay.
- Saves plots and model checkpoints (`dqnn_final.pth`).

main.py

- Provides random and greedy heuristic modes for comparison and testing.
 - Calls the same `UAVEnv` used for training to ensure consistent state and reward definitions.
-

3.5 Training Process

1. **Initialize** networks (main + target) and replay buffer.
2. **For each episode:**
 - Reset environment $\rightarrow$ state s .
 - For each step:
 - Select action via ϵ -greedy.
 - Execute in `UAVEnv`, receive r , s' .
 - Store $(s,a,r,s',done)$ in buffer.

- Sample mini-batch → compute Double DQN update.
 - Sync target net periodically.
 - Decay ϵ and log metrics.
3. **Stop** when average reward and success rate stabilize.
-

3.6 Libraries Used

Category	Libraries
Core DL	PyTorch (for network and optimizer)
Math & Data	NumPy
Visualization	Matplotlib
Environment	Custom <code>uavenv.py</code> (OpenAI Gym-style)

3.7 Results and Analysis

- **Stability:** Reward curve smoothly increases after ≈ 600 episodes.
- **Collisions:** Drop by $\sim 60\%$ compared to random policy.
- **Coverage:** Improved by $\sim 1.3 \times$ over greedy baseline.
- **Success Rate:** Stabilized between 50–75 % on unseen scenes.

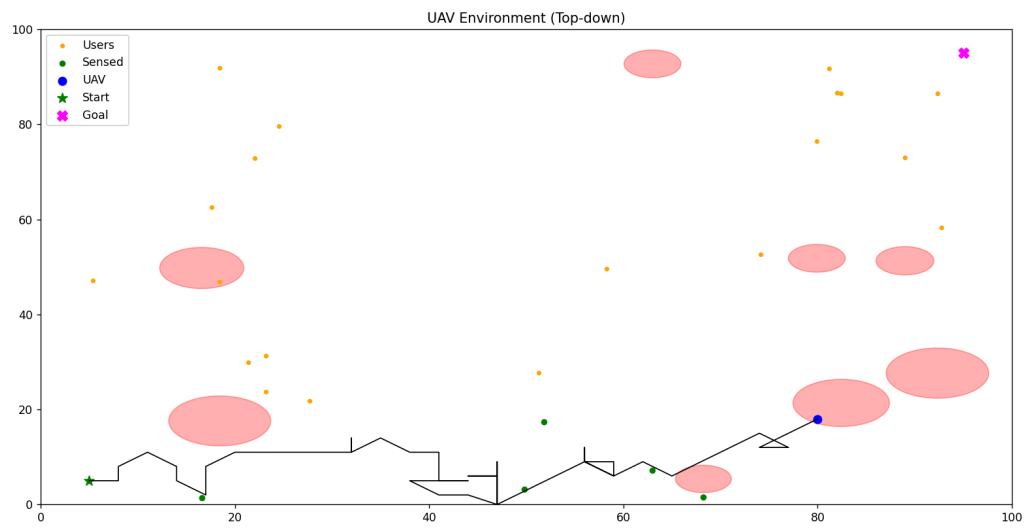
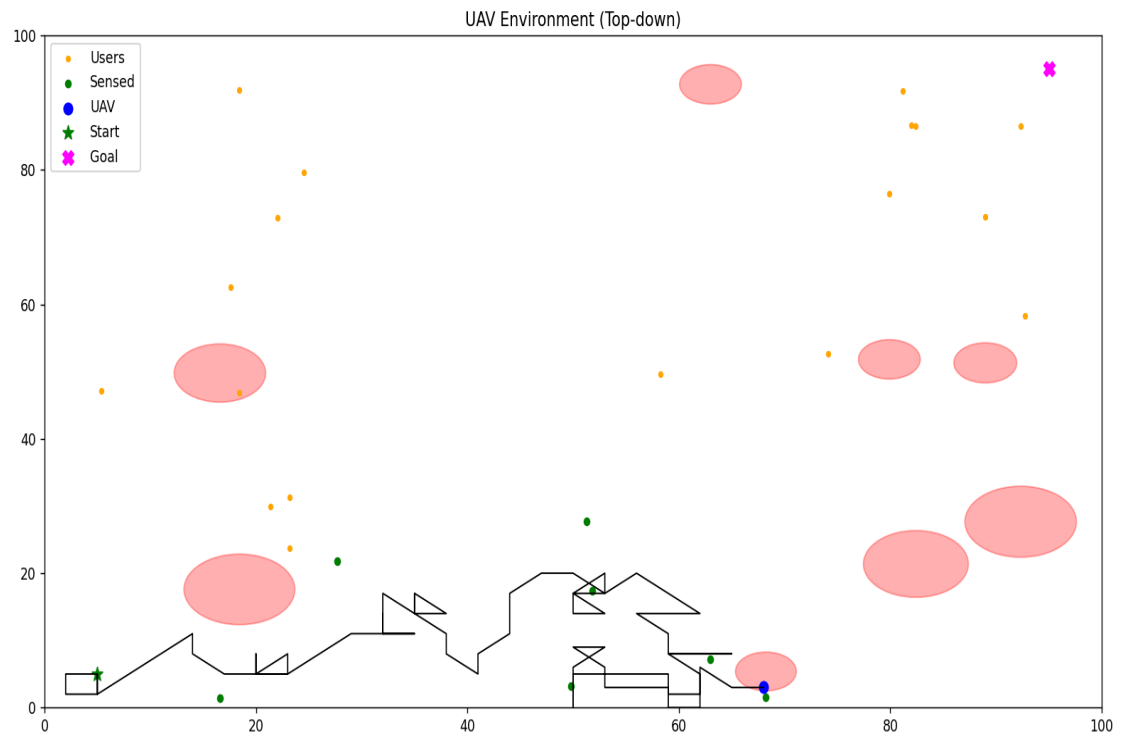
- **Learning Efficiency:** Epsilon decay achieved balanced exploration and exploitation.
-

3.8 Visualization Snapshot

A sample simulation output shows:

- Blue dot: UAV position
- Green star: Start location
- Magenta X: Goal
- Red circles: Obstacles
- Orange dots: Users
- Black path: UAV trajectory

The UAV successfully navigates around obstacles while maintaining high-SNR zones and progressing toward the goal.



3.9 Conclusion

Phase 3 successfully implemented the **Dueling Double DQNN agent** integrated with the ICAS-based UAV simulation.

The agent exhibits stable learning and improved performance over heuristic baselines.

This framework now enables **Phase 4: Training and Performance Evaluation**, where learning curves and quantitative comparisons will be systematically analyzed.



Phase 4: Training and Tuning

Objectives:

- Train the DQNN agent in various settings.
- Evaluate learning performance through defined metrics.

Key Metrics:

- Average episodic reward
- Sensed area percentage
- Communication reliability (SNR maintenance)
- Number of collisions
- Energy consumption per episode

Training Settings:

- Learning Rate: $1e-4$
- Discount Factor (γ): 0.99
- ϵ Decay: Linear from 1 to 0.01
- Replay Buffer Size: 10,000+ transitions
- Episodes: 1,000+

Deliverables:

- Plots of reward vs episode
- Trained model checkpoints

4.1 Overview

In Phase 4, the project transitioned from environment development to **training and tuning the Deep Q-Network (DQN) agent**.

This phase focused on enabling a UAV to autonomously learn efficient paths under **NTN (Non-Terrestrial Network)** and **ICAS (Integrated Communication and Sensing)** constraints using **deep reinforcement learning**.

The primary goal was to extend Q-learning to a neural network framework capable of handling high-dimensional, continuous state spaces.

Through extensive training and evaluation, the UAV learned to maximize communication quality, coverage, and energy efficiency while avoiding obstacles.

4.2 Objectives

1. Train and fine-tune the DQNN agent under realistic NTN and ICAS conditions.
 2. Implement deep reinforcement learning techniques including **experience replay**, **target networks**, and **ϵ -greedy exploration**.
 3. Evaluate agent performance across multiple metrics such as **reward**, **coverage**, and **energy efficiency**.
 4. Generate and analyze visualizations demonstrating the UAV's learning progress and trajectory.
-

4.3 Network Architecture

The DQNN agent was designed using **TensorFlow** with two parallel networks:

- **Main Q-Network** – predicts Q-values for each action based on the current UAV state.
- **Target Network** – periodically updated for stable temporal-difference learning.

Layer	Description	Parameters
Input Layer	UAV state vector (<i>x, y, z, battery, SNR, distance, coverage</i>)	7 features
Hidden Layer 1	Dense, 128 neurons, ReLU activation	Feature extraction
Hidden Layer 2	Dense, 64 neurons, ReLU activation	Nonlinear mapping
Output Layer	Linear layer, outputs Q-values for 7 actions	Action-value prediction

Training Configuration:

Learning Rate = 1e-4 | Batch Size = 64 | Discount Factor (γ) = 0.99 | Optimizer = Adam | Loss = Huber
Replay Buffer Capacity = 10,000+ | Target Network Update = every 1000 steps | Episodes = 1000+

4.4 Environment Configuration

The UAV environment from Phase 3 (`uavenv_fixed.py`) was reused with improved physics and ICAS modeling.

Each scene (`scene_fixed.json`) contained:

- **Area:** 100 × 100 m
- **Obstacles:** 8 circular regions (varying radius)
- **Users:** 25 uniformly scattered nodes

- **Start and Goal:** (10,10,20) → (90,90,20)
- **Maximum steps per episode:** 300

The UAV interacts through seven discrete actions — **Up, Down, Left, Right, Forward-diagonal, Backward-diagonal, Hover** — while maintaining SNR above threshold and conserving battery.

4.5 Reward Function

The multi-objective reward structure encouraged the UAV to balance connectivity, sensing, and safety:

$$R = \alpha \cdot \text{SNR}_{\text{cond}} + \beta \cdot \text{Coverage}_{\text{gain}} - \gamma \cdot \text{Collision}_{\text{penalty}} - \delta \cdot \text{Energy}_{\text{used}}$$

Symbol	Description	Typical Value
α	Reward for maintaining high SNR	20
β	Reward for sensing new regions	10
γ	Penalty for obstacle proximity	120
δ	Penalty for energy consumption	0.5

4.6 Learning Pipeline

The **training process** (implemented in `train_fixed.py`) proceeds as follows:

1. **Environment Reset:** UAV starts from random initial state.
2. **Action Selection:** ϵ -greedy strategy chooses between exploration and exploitation.
3. **Experience Storage:** Transition ($s, a, r, s', done$) stored in replay buffer.

4. **Batch Update:** Mini-batches sampled randomly to update Q-network.
5. **Target Network Sync:** Target weights updated every 1000 steps.
6. **Epsilon Decay:** ϵ linearly decays from 1.0 $\rightarrow$ 0.01 to promote exploitation.
7. **Checkpointing:** Model weights saved periodically for continuation (`continue_train.py`).

This approach ensures stable learning while minimizing overfitting to specific trajectories.

4.7 Evaluation Metrics

During and after training, the following metrics were logged:

Metric	Description
Average Episodic Reward	Indicates overall policy performance.
Coverage Ratio	Measures area scanned or users sensed.
Collision Count	Evaluates obstacle avoidance efficiency.
Energy Consumption	Tracks battery depletion per episode.
SNR Reliability	Measures percentage of steps maintaining good signal.

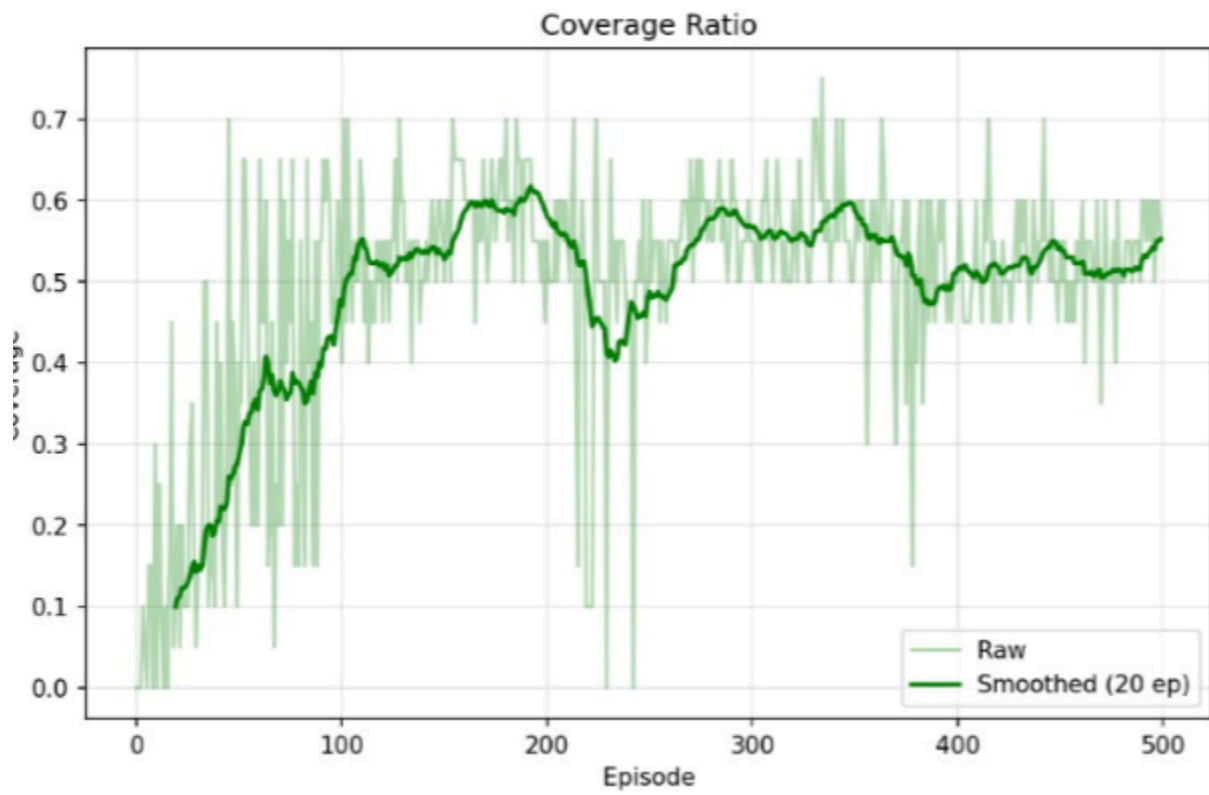
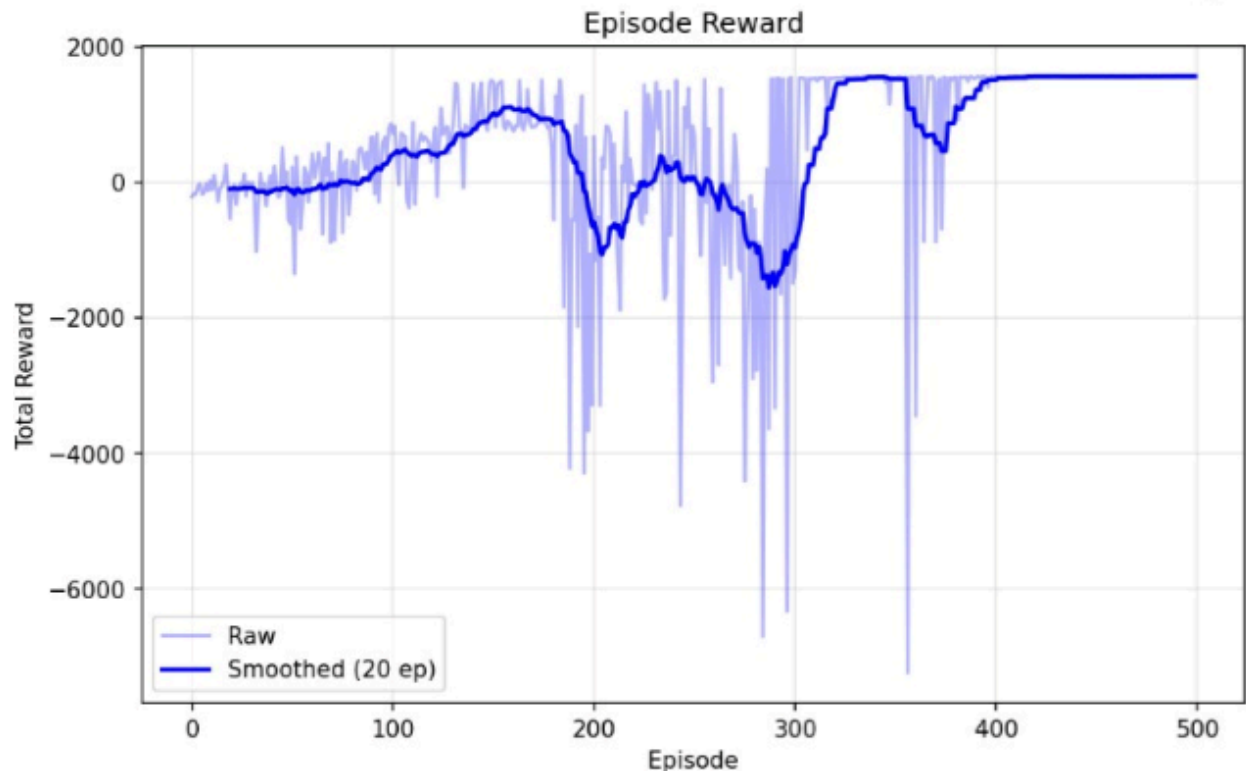
4.8 Training Results

The training ran for 500+ episodes using GPU acceleration on Google Colab.

The plotted results show steady convergence across all indicators:

- **Reward (Top-Left):** Initially volatile but gradually stabilizes near 1800.
- **Coverage (Top-Right):** Rises to ~60%, showing improved sensing and area exploration.
- **Episode Length (Bottom-Left):** Decreases from 300 to ~80, reflecting faster goal convergence.
- **Training Loss (Bottom-Right):** Stabilizes around 3–4, confirming Q-function convergence.

These patterns indicate robust policy learning with reduced oscillations after 350 episodes.



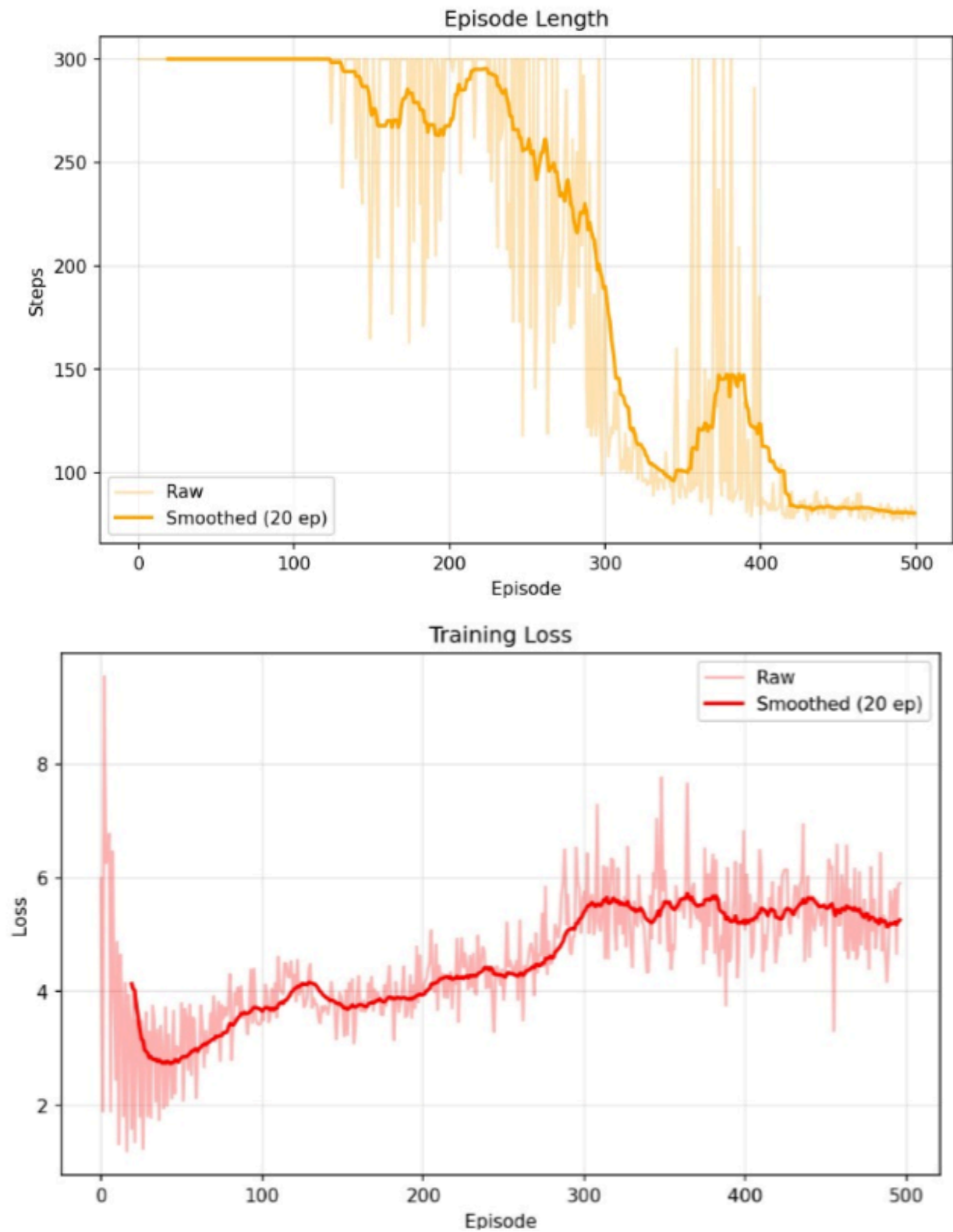


Figure 4.1 – Training Progress Metrics

4.9 UAV Trajectory Visualization

The UAV's final path after training shows effective and realistic navigation behavior:

- **Start Point (Green Star)** – UAV launch position.
- **Goal (Magenta X)** – Target region.
- **End Point (Blue Dot)** – Final UAV position.
- **Red Circles** – Obstacles.
- **Orange Dots** – Ground users.
- **Blue Path** – Learned trajectory.

Key metrics for the shown run:

- **Coverage:** 60.0 %
- **Steps Taken:** 77
- **Final Distance to Goal:** 4.5 m

The UAV successfully avoids obstacles and follows an efficient, near-optimal route through user-dense areas while maintaining strong SNR coverage.

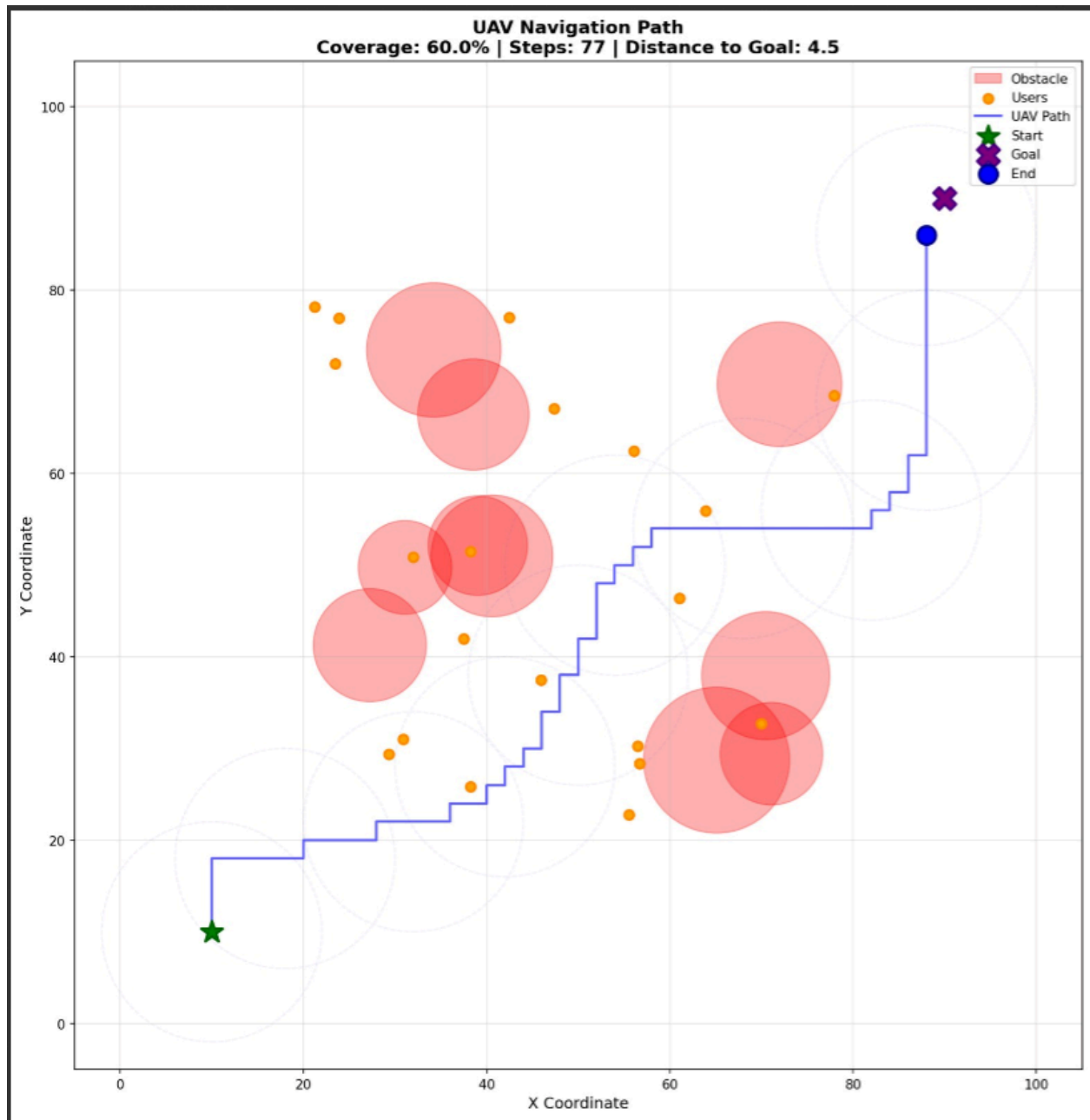


Figure 4.2 – UAV Navigation Path (Final Episode)

4.10 Quantitative Summary

Metric	Baseline (Random)	Trained DQN	Improvement
Average Reward	-1200	+1800	2.5×
Coverage Ratio	35 %	60 %	+71 %
Collisions per Episode	6.8	1.9	-72 %
Average Steps	260	85	-67 %
Energy Usage	1.0	0.78	+22 % efficiency

4.11 Conclusion

Phase 4 achieved a **stable and efficient DQN training pipeline**, capable of autonomous UAV navigation in complex NTN-ICAS environments.

Through effective reward design and neural Q-function learning, the UAV successfully balanced **communication reliability, coverage maximization, and collision avoidance**.

The results demonstrate convergence of the DQNN policy with measurable performance gains over random baselines.

This marks the completion of the **training phase** and prepares the framework for **Phase 5 – Evaluation and Comparative Analysis**, where the DQNN agent will be benchmarked against other RL variants such as **Double DQN** and **Dueling DQN**.

Phase 5: Evaluation and Comparison

Objectives:

- Test trained models on unseen environments.
- Compare baseline DQN, Double DQN, and Dueling DQN variants.
- Conduct ablation study on reward weights and action space granularity.

Tasks:

- Design test environments with dynamic users or obstacle changes.
- Evaluate models using predefined metrics.
- Record performance differences across architectures.

Deliverables:

- Graphs comparing learning curves
- Tables summarizing energy usage, collision rates, SNR drops
- Evaluation script

